Where Do OpenMP Threads Help? A Stage-by-Stage Performance Analysis of a Multi-Scale Frangi Pipeline for Retinal Vessel Segmentation

Devik Nangia, Snehal Tyagi, Srijan Bera, and Prathamesh Patil

Manipal Institute of Technology, Manipal, India

Abstract. Diabetic retinopathy screening depends on fast and reliable segmentation of retinal vessels, and the multi-scale Frangi vesselness filter is a training-free classical method whose cost grows with image resolution and with the number of scales. We implement a complete pipeline (green-channel extraction, CLAHE, multi-scale Frangi filtering, thresholding and morphological clean-up) in C, parallelise every stage with OpenMP, and compare it stage by stage with a pragma-free serial build of the same code on a 2-core/4-thread cloud CPU. Serial and OpenMP outputs are bit-identical on every DRIVE and STARE image for all thread counts and design variants. With parameters tuned on the DRIVE training images only, the pipeline reaches a mean Dice of **[DICE-DRIVE]** on the DRIVE test set and **[DICE-STARE]** on STARE. The whole pipeline speeds up by **[S4]**× with 4 threads (efficiency **[E4]**), while 2 threads placed on the two hyperthreads of one core give only **[S2-close]**× against **[S2-spread]**× when spread over both cores. The Gaussian-derivative stage dominates serial time (**[GAUSS-SHARE]**%); morphology scales worst. Owner-computes tile histograms beat atomic histograms in CLAHE by **[ATOMIC-RATIO]**×, and running the four Frangi scales concurrently is **[SCALEPAR-RATIO]**× slower than parallelising inside each scale because of load imbalance. Speedup stays between **[SMIN]** and **[SMAX]** from 0.33 to 21.1 megapixels. We report Karp–Flatt serial fractions per stage and throughput in images per second, and place the accuracy beside published lightweight deep models.

Keywords: Parallel computing · OpenMP · Multicore · Hyper-Threading · Frangi filter · CLAHE · Retinal vessel segmentation · Diabetic retinopathy

1 Introduction

Diabetic retinopathy damages the small blood vessels of the retina and is a leading preventable cause of vision loss. It is diagnosed from colour fundus photographs, and the first computational step in most automated screening systems is segmenting the retinal vessels [5]. India alone had an estimated 89.8 million adults with diabetes in 2024 [9]. Screening camps capture fundus images in bulk, often on modest hardware and with few specialists available to read them, so throughput (images per second) is a practical requirement.

The multi-scale Frangi vesselness filter [4] is a widely used classical method for this step. It needs no training data, behaves deterministically and can be written in plain C.

Its cost grows with image resolution and with the number of scales, because every pixel needs several Gaussian-derivative convolutions followed by an eigenvalue computation. A complete screening pipeline adds contrast enhancement before the filter and binary clean-up after it, and these stages stress a processor in different ways: convolution is memory-bound, the eigenvalue step is arithmetic-heavy, CLAHE builds histograms, and morphology moves single bytes.

Multicore CPUs are the hardware every screening site already has. A developer choosing how to parallelise such a pipeline needs to know, stage by stage, where OpenMP threads help, where they stop helping, and why. Published parallel retinal work has used other algorithms or isolated single stages [1,2], and the closest cross-platform study covers a single enhancement filter [17].

This paper implements the full pipeline (green-channel extraction, CLAHE, multi-scale Frangi filtering, thresholding and morphological clean-up) in C, parallelises every stage with OpenMP, and analyses its performance stage by stage against a pragma-free serial build of the same code. Accuracy is measured on the DRIVE benchmark [18], with STARE [8] as a cross-dataset check, using parameters tuned on the DRIVE training images only. The contributions are:

- a documented serial/OpenMP C implementation whose outputs are bit-identical across all thread counts and design variants, verified on every test image;
- a stage-by-stage measurement of time, speedup, efficiency and Karp–Flatt serial fraction on a 2-core/4-thread cloud CPU, including the effect of Hyper-Threading and thread placement;
- an ablation of three alternative OpenMP designs (dynamic tile scheduling, atomic histograms and scale-level parallelism) and a resolution-scaling study from 0.33 to 21.1 megapixels;
- segmentation accuracy on DRIVE and STARE reported alongside published lightweight deep models.

The study tests four hypotheses. H1 and H4 are taken unchanged from the project proposal. The proposal's two GPU hypotheses fall outside this OpenMP-only study and are replaced by H2 and H3, which were written after a short pilot run (two images, three repetitions) used to debug the harness; the pilot data are not reported. **H1**: OpenMP speedup grows sub-linearly with thread count and is limited by the number of physical cores and by memory bandwidth. **H2**: OpenMP speedup is stable or grows with image size, because the fixed costs of creating parallel regions shrink relative to the work. **H3**: the best decomposition differs by stage: owner-computes tile histograms beat atomic histograms in CLAHE, and parallelism inside each scale beats running the scales concurrently in the Frangi stage, because the four scales have unequal cost. **H4**: the classical pipeline does not match the best lightweight deep models in Dice (for example, 0.8070 on DRIVE for LightVesselNet [16]); its value is that it is training-free and its parallel behaviour can be explained.

Section 2 reviews related work, Section 3 describes the pipeline, the OpenMP design and the measurement protocol, Section 4 reports the results and Section 5 discusses them against the hypotheses.

2 Related Work

2.1 Surveys and accuracy-centred work

Fraz et al. [5] categorise classical retinal vessel extraction methods, including Hessian-based vesselness, and compare them on DRIVE [18] and STARE [8]. Later reviews concentrate on deep networks: Chen et al. [3] survey architectures and open problems, Sule [19] organises 110 papers from 2016–2021 by the technique used to improve accuracy, and Gojić et al. [7] summarise recent methods with their reported metrics. Execution time on commodity hardware is not an organising axis in any of these surveys.

2.2 Classical vesselness methods

The Frangi filter [4] scores each pixel by how tube-like its Hessian eigenvalue structure is, over several Gaussian scales. Mahapatra et al. [11] keep this interpretable core, tune its parameters with particle-swarm optimisation and follow it with fuzzy clustering. The method is unsupervised, but the optimisation and the clustering add computation, and parallel execution is not studied.

2.3 Lightweight deep models

Galdran et al. [6] show that minimalistic U-Nets approach state-of-the-art accuracy and that cross-dataset generalisation remains weak. Deployment-oriented networks report model size instead of measured time: LVS-Net [13] (0.71 M parameters, 29.60 GFLOPs, DRIVE Dice 0.8644), LFA-Net [14] (0.11 M parameters, 4.46 GFLOPs, DRIVE Dice 0.8328) and LightVesselNet [16] (75 K parameters, Dice 0.8070 on DRIVE and 0.8072 on STARE). These models need annotated training data, trained weights and a deep-learning runtime.

2.4 Parallel and accelerated vessel analysis

Argüello et al. [1] map a filtering-plus-contour-tracing method to CUDA and segment a DRIVE image in 14 ms on a GTX 680. Ben Sayadia et al. [2] accelerate the Hessian eigenvalue step on a GPU and report a 27× speedup over a multithreaded CPU version, applied to thick-vessel segmentation. Nahar et al. [15] move Hessian-based filtering of 3-D mitochondrial images to the GPU (up to 11× and 30×) and note that small inputs under-use the device. Song et al. [17] compare serial, OpenMP, CUDA and OpenCL versions of unsharp masking over image sizes from 525^2 to $16{,}364 \times 8{,}182$ and report OpenMP speedups that track the core count. That study is the closest methodological template for ours, but it covers a single enhancement filter and no vessel analysis.

2.5 Gap

We found no study that parallelises a complete CLAHE–Frangi–morphology pipeline on a multicore CPU and profiles every stage. Prior parallel retinal work either uses a different algorithm [1] or a single stage [2]. Generic comparisons [17] cover one filter

whose behaviour may not transfer, because the Frangi pipeline mixes a memory-bound convolution stage, an arithmetic-heavy eigenvalue stage, a histogram stage and byte-wide morphology.

3 Method

The serial and the OpenMP builds are compiled from one C source file set. The serial build omits `-fopenmp`, so every pragma is ignored and the same loops run on one thread. Any difference in time is therefore due to parallel execution, not to a different algorithm.

3.1 Pipeline

Stage 1, green channel. $G(x, y)$ is taken from the RGB fundus image, where vessels have the highest contrast. DRIVE images are LZW-compressed TIFF files, which we decode with our own C reader (verified pixel-exact against Pillow); STARE images are PPM files.

Stage 2, CLAHE [22]. The image is split into an 8×8 grid of tiles of $\lceil w/8 \rceil \times \lceil h/8 \rceil$ pixels. Each tile's 256-bin histogram is clipped at $\kappa N_t/256$ counts (N_t tile pixels, κ the clip factor), the excess is spread uniformly over the bins, and the cumulative histogram gives a lookup table. Each output pixel is a bilinear blend of the four nearest tile tables.

Stage 3, multi-scale Frangi filter [4]. With I the CLAHE output scaled to $[0, 1]$, the second derivatives at scale σ are computed by separable 1-D convolutions with sampled Gaussian kernels of radius $r = \lceil 3\sigma \rceil$: $I_{xx} = g_\sigma^{(y)} * (g_\sigma^{\prime\prime(x)} * I)$, $I_{yy} = g_\sigma^{\prime\prime(y)} * (g_\sigma^{(x)} * I)$ and $I_{xy} = g_\sigma^{\prime(y)} * (g_\sigma^{\prime(x)} * I)$, with replicated borders. The kernels are normalised so that $\sum g = 1$, $\sum k\, g'(k) = -1$, $\sum g'' = 0$ and $\sum \frac{1}{2} k^2 g''(k) = 1$. One row sweep produces the three row-filtered images and one column sweep the three derivatives, i.e. six 1-D passes per scale. The scale-normalised Hessian $\sigma^2\mathbf{H}$ has eigenvalues

$$\lambda_{1,2} = \tfrac{1}{2}(a + d) \mp \sqrt{\tfrac{1}{4}(a - d)^2 + b^2}, \quad a = \sigma^2 I_{xx},\ b = \sigma^2 I_{xy},\ d = \sigma^2 I_{yy}, \tag{1}$$

relabelled so that $|\lambda_1| \le |\lambda_2|$. Vessels are dark on a bright background, so the response is non-zero only when $\lambda_2 > 0$:

$$\mathcal{V}_\sigma = \exp\!\left(-\frac{R_B^2}{2\beta^2}\right)\left[1 - \exp\!\left(-\frac{S^2}{2c^2}\right)\right], \quad R_B = \frac{|\lambda_1|}{\lambda_2},\ S^2 = \lambda_1^2 + \lambda_2^2, \tag{2}$$

and $V(x, y) = \max_{\sigma \in \{1,2,3,4\}} \mathcal{V}_\sigma(x, y)$.

Stage 4, threshold and morphology. V is thresholded at a global value τ and multiplied by the field-of-view (FOV) mask eroded by e pixels, which removes the bright ring that the filter produces on the FOV border. A 3×3 binary opening (erosion then dilation) removes speckle and a 3×3 closing bridges one-pixel gaps.

Stage 5, evaluation. True and false positives and negatives are counted inside the (uneroded) FOV and converted to Dice $= 2TP/(2TP + FP + FN)$, sensitivity, specificity and accuracy.

Table 1. Parallel decomposition by stage.

Stage	OpenMP decomposition
Green channel	`parallel for` over pixels
CLAHE	`parallel for` over the 64 tiles, each thread owning its tile histogram and lookup table; `parallel for` over rows for the bilinear blend
Gaussian derivatives	row sweep: `parallel for` over rows with a thread-private padded row buffer; column sweep: `parallel for` over output rows, inner loop over x vectorised
Eigen + vesselness	`parallel for` over pixels, fused with the running maximum over scales
Threshold	`parallel for` over pixels, fused with the FOV mask
Morphology	four 3×3 passes, each a `parallel for` over rows
Metrics	`reduction(+:TP,FP,TN,FN)`

3.2 Parameter selection

All parameters are chosen on the 20 DRIVE training images only, by maximising the mean per-image Dice over a grid: $\kappa \in \{1, 2, 3, 4\}$, $\beta \in \{0.5, 1\}$, $c \in \{0.005, 0.01, 0.02, 0.04, 0.08, 0.16\}$, $e \in \{4, 8, 12\}$ and 40 thresholds spaced logarithmically in $[10^{-3}, 0.9]$ (5,760 settings). The selected setting is then fixed and applied unchanged to the 20 DRIVE test images, scored against the first observer's manual segmentation, and to the 20 STARE images, scored against Hoover's labels [8]. STARE has no FOV masks, so the FOV is the set of pixels whose maximum channel value exceeds 35, cleaned by a 3×3 opening.

3.3 OpenMP design

Table 1 lists the parallel decomposition of each stage. All loops use `schedule(static)` unless stated, and each output element is computed by exactly one thread, so floating-point summation order is the same as in the serial build. Three alternative designs are kept as ablations: a dynamic schedule for the CLAHE tile loop, a pixel-parallel CLAHE histogram with `#pragma omp atomic` increments, and a Frangi decomposition that runs the four scales concurrently with one thread each instead of parallelising inside each scale.

3.4 Correctness checks

For every test image the serial build writes the CLAHE output, the vesselness map V and the final mask, and every OpenMP configuration (each design variant at 1, 2, 4 and 8 threads) writes the same files. We report the fraction of identical CLAHE bytes, identical mask pixels and bit-identical V values, and the maximum $|\Delta V|$. The comparison is repeated for builds with floating-point contraction disabled (`-ffp-contract=off`). As an external check, V from the serial build is compared with `skimage.filters.frangi` [20] applied to the same CLAHE output with the same σ, β and c; the two implementations normalise derivatives differently, so we report correlation rather than equality.

Table 2. Serial vs OpenMP equivalence on the DRIVE test set (fractions over all pixels).

Build / variant / p	CLAHE equal	Mask equal	V bit-equal	max $\lvert\Delta V\rvert$
FMA, base, 1–8	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**
FMA, dynamic, 1–8	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**
FMA, atomic, 1–8	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**
FMA, scale-parallel, 1–8	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**
no-FMA, all variants, 1–8	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**
STARE, base, 4	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**

3.5 Platform and measurement protocol

All timed runs use one Kaggle CPU notebook: an Intel Xeon at 2.20 GHz exposing 4 logical CPUs (2 physical cores with 2-way Hyper-Threading [12]), 32 KiB L1d and 256 KiB L2 per core, a 55 MiB L3 and 31 GiB RAM, with a cgroup quota of 4 CPUs, running Linux 6.18 with GCC 13.3 (`-O3 -march=native`, OpenMP 4.5). Nothing is timed on the authors' laptops. Thread counts are 1, 2, 4 and 8 (the last oversubscribes the 4 logical CPUs); threads are bound with `OMP_PROC_BIND=close` and `OMP_PLACES=threads` unless stated. Each configuration runs 5 untimed warm-up iterations and 30 timed iterations per image over all 20 DRIVE test images. Time is measured with `omp_get_wtime` (OpenMP build) or the monotonic clock (serial build) around each stage; file I/O and image decoding are excluded. Speedup is $S(p) = T_{\text{serial}}/T(p)$ with T_{serial} from the pragma-free build, efficiency is $E = S/p$ and the Karp–Flatt serial fraction [10] is $e = (1/S - 1/p)/(1 - 1/p)$. Throughput is images per second for one image at a time.

For resolution scaling, three test images are bilinearly resampled to 2×, 4× and 8× linear size (0.33 to 21.1 megapixels) before timing, with the FOV mask resampled by nearest neighbour and the erosion radius scaled accordingly; the number of timed iterations is reduced at large sizes (10, 10, 5 and 3). A thread-placement experiment repeats the thread sweep with one thread per physical core first (`OMP_PLACES=cores`, `OMP_PROC_BIND=spread`) and with binding disabled.

4 Results

4.1 Correctness

Table 2 summarises the equivalence checks over the 20 DRIVE test images. **[Fill from verify_*.csv: fraction of identical CLAHE bytes, mask pixels and bit-identical V for every variant and thread count, FMA and no-FMA builds, plus STARE.]** The serial FMA and no-FMA builds **[do / do not]** produce identical V (**[max $\lvert\Delta V\rvert$]**). Our TIFF decoder reproduces Pillow's green channel exactly on the three images checked. Against `skimage.filters.frangi` on the same CLAHE input, the Pearson correlation of V inside the FOV is **[r]** and the Spearman correlation is **[rho]**.

Table 3. Segmentation accuracy inside the FOV (mean over 20 images; Dice ± s.d.).

Set	Dice	Sens.	Spec.	Acc.
DRIVE test	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**
DRIVE train (tuning)	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**
STARE (cross-dataset)	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**
LightVesselNet [16], DRIVE	0.8070	0.8189	–	–
LightVesselNet [16], STARE	0.8072	0.8499	–	–
LFA-Net [14], DRIVE	0.8328	–	–	–
LVS-Net [13], DRIVE	0.8644	–	–	–

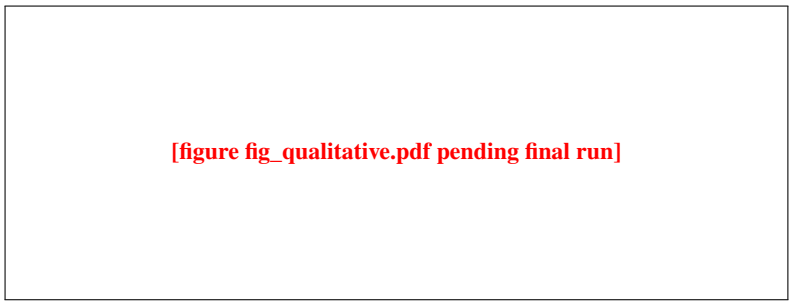


Fig. 1. DRIVE test images 01 and 02: input, vesselness V, and the final mask compared with the manual segmentation (true positives black, false positives orange, false negatives blue).

4.2 Segmentation accuracy

The grid search selected $\kappa =$ **[TBD]**, $\beta =$ **[TBD]**, $c =$ **[TBD]**, $e =$ **[TBD]** and $\tau =$ **[TBD]** (mean training Dice **[TBD]**). Table 3 gives the accuracy with these settings fixed.

Figure 1 shows two test images with their vesselness maps and the errors of the final mask. **[One or two sentences on where errors concentrate: thin capillaries missed, optic-disc rim and lesions as false positives.]**

4.3 Serial profile

Table 4 breaks the serial time per image into stages. **[The Gaussian-derivative passes take X% and the eigenvalue/vesselness stage Y%; CLAHE and morphology together take Z%.]**

4.4 Thread scaling

Table 5 and Fig. 2 give whole-pipeline and per-stage scaling. **[Speedup at** $p = 1, 2, 4, 8$**; efficiency; Karp–Flatt** e**; throughput in images/s. Note the OpenMP build at** $p = 1$ **against the serial build, the small gain at** $p = 2$ **with close binding, the jump at** $p = 4$**, and the plateau or loss at** $p = 8$**.]**

Table 4. Serial time per DRIVE test image by stage (20 images × 30 runs).

Stage	Mean (ms)	S.d.	Median	Share (%)
Green channel	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**
CLAHE	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**
Gaussian derivatives	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**
Eigen + vesselness	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**
Threshold	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**
Morphology	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**
Total	**[TBD]**	**[TBD]**	**[TBD]**	100

Table 5. Whole-pipeline scaling on DRIVE test images (close binding, `OMP_PLACES=threads`).

p	Time (ms)	S	E	e	Images/s
serial	**[TBD]**	1.00	–	–	**[TBD]**
1	**[TBD]**	**[TBD]**	**[TBD]**	–	**[TBD]**
2	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**
4	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**
8	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**

4.5 Thread placement

Table 7 repeats the sweep on five images with three placements. **[With one thread per physical core first, $p = 2$ reaches X× instead of Y×; at $p = 4$ all placements coincide because every logical CPU is used.]**

4.6 Alternative OpenMP designs

Table 8 compares each ablation with the base design on the stage it changes. **[Dynamic scheduling of the 64 CLAHE tiles costs/gains X%; atomic histograms are Y× slower; running the scales concurrently is Z× slower at $p = 4$.]**

4.7 Resolution scaling

Table 9 and Fig. 4 show how speedup changes with image size. **[Serial time grows from X ms at 0.33 MP to Y s at 21.1 MP; speedup at the best thread count stays between A and B; throughput in megapixels per second.]**

4.8 Cross-dataset timing

On the 20 STARE images (700 × 605) the serial pipeline takes **[TBD]** ms per image and reaches **[TBD]**× at $p = 4$, **[consistent with / different from]** DRIVE.

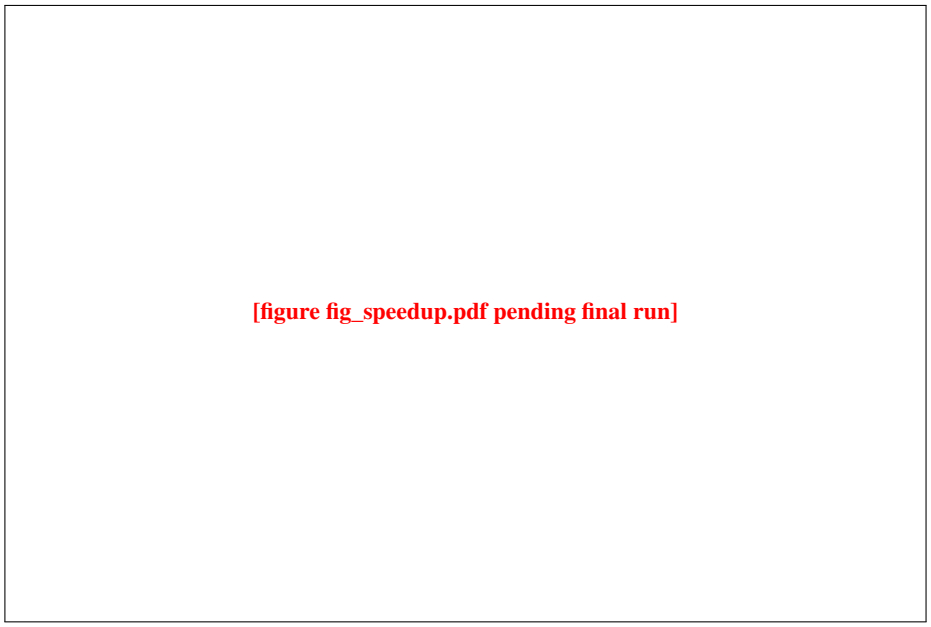


Fig. 2. Speedup over the serial build per stage and for the whole pipeline; dashed line is ideal.

5 Discussion

5.1 Hypotheses

H1 (sub-linear scaling, limited by physical cores). **[Supported / not supported.]** The machine has two physical cores. **[With close binding, two threads share one core and give only X×; spreading them over both cores gives Y×. Going from 2 to 4 threads adds the second hyperthread on each core and raises the speedup to Z×, a Hyper-Threading gain of W% that comes from overlapping memory stalls in the convolution and the exponentials in the vesselness stage [12]. Oversubscription to 8 threads adds no gain.]** The Karp–Flatt fraction **[rises / stays flat]** with p, which indicates **[growing overhead (thread contention, shared caches) rather than a fixed serial part]** [10].

H2 (speedup stable or growing with image size). **[Supported / not supported: speedup ranges from A to B between 0.33 and 21.1 megapixels.]** At native size each parallel region does about a third of a megapixel of work, so fork–join overhead is small compared with the Gaussian passes; **[at large sizes the working set (six float images of 4 B per pixel) exceeds the 55 MiB L3 above roughly 2.3 megapixels, and the column passes become bound by memory bandwidth [21].]**

H3 (best decomposition differs by stage). **[Supported / not supported.]** Owner-computes tile histograms need no synchronisation, whereas the atomic variant serialises updates to popular bins of the same tile. **[Dynamic scheduling of the 64 tiles changes little because tile costs are nearly equal.]** Running the four scales concurrently is lim-

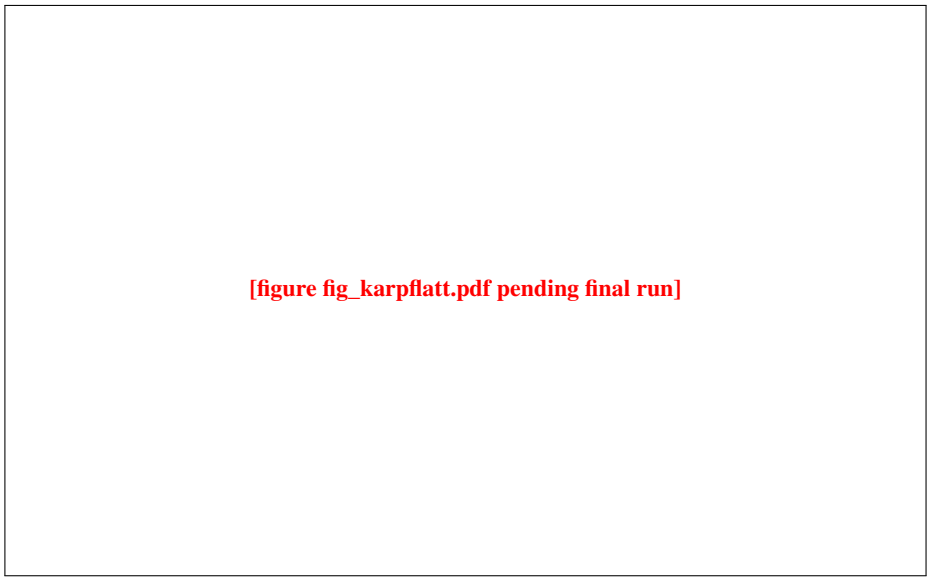


Fig. 3. Karp–Flatt serial fraction e per stage.

Table 6. Per-stage speedup over the serial build.

Stage	$p = 1$	$p = 2$	$p = 4$	$p = 8$
Green channel	[TBD]	[TBD]	[TBD]	[TBD]
CLAHE	[TBD]	[TBD]	[TBD]	[TBD]
Gaussian derivatives	[TBD]	[TBD]	[TBD]	[TBD]
Eigen + vesselness	[TBD]	[TBD]	[TBD]	[TBD]
Threshold	[TBD]	[TBD]	[TBD]	[TBD]
Morphology	[TBD]	[TBD]	[TBD]	[TBD]
Total	[TBD]	[TBD]	[TBD]	[TBD]

ited by the largest scale: the $\sigma = 4$ kernels have $2 \cdot 12 + 1 = 25$ taps against 7 for $\sigma = 1$, so one thread carries most of the work while the others idle.

H4 (accuracy below lightweight deep models). [Supported:] the DRIVE test Dice of [TBD] is below 0.8070 for LightVesselNet [16], 0.8328 for LFA-Net [14] and 0.8644 for LVS-Net [13]. The pipeline needs no annotated data, no weights and no deep-learning runtime, and its parameters transfer to STARE without retuning (Dice [TBD]). The deep-model figures come from the authors' own papers and protocols, so the comparison is indicative.

5.2 Practical implications

[At X images per second on a 2-core VM, one such machine processes a screening camp's daily intake of N images in M minutes.] Because the Gaussian-derivative stage

Table 7. Effect of thread placement (5 DRIVE test images).

Placement	$S(2)$	$S(4)$	$S(8)$
close, threads	**[TBD]**	**[TBD]**	**[TBD]**
spread, cores	**[TBD]**	**[TBD]**	**[TBD]**
no binding	**[TBD]**	**[TBD]**	**[TBD]**

Table 8. Design ablations: stage time relative to the base design (ratio > 1 is slower).

Variant	Stage	$p = 1$	$p = 2$	$p = 4$	$p = 8$
dynamic tiles	CLAHE	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**
atomic histogram	CLAHE	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**
scale-parallel	Frangi	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**

dominates, a developer should parallelise it first; morphology and thresholding are cheap enough that their poor scaling hardly affects the total. Thread placement matters as much as thread count on small SMT machines: an unbound or close-bound run with fewer threads than logical CPUs can leave a whole core idle.

5.3 Limitations

The study uses a single cloud VM with two physical cores, shared with other tenants, so results for larger core counts and for NUMA systems are not covered, and run-to-run variance is higher than on a dedicated machine; we report standard deviations and medians for this reason. The CPU model is reported only as "Intel Xeon @ 2.20 GHz". GPU implementations, which the original proposal planned, are outside the scope of this paper. The accuracy pipeline uses one global threshold and a fixed FOV erosion, and STARE FOV masks are computed by thresholding rather than supplied with the dataset.

6 Conclusion

We parallelised a complete CLAHE–Frangi–morphology retinal vessel pipeline with OpenMP and measured it stage by stage against a pragma-free serial build of the same C code. The parallel version produces bit-identical results, reaches **[S4]**× on two physical cores with Hyper-Threading and processes **[TBD]** DRIVE images per second, with accuracy (Dice **[TBD]** on DRIVE, **[TBD]** on STARE) below lightweight deep models but obtained without training. The Gaussian-derivative passes dominate the cost; thread placement on SMT cores, owner-computes histograms and parallelism inside each Frangi scale are the design decisions that matter most. Future work covers machines with more cores, explicit cache blocking of the column passes, and the GPU comparison planned in the original proposal.

Table 9. Resolution scaling (3 DRIVE test images, best thread count).

Scale	MP	Serial (ms)	OpenMP (ms)		S	MP/s
1×	0.33	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**
2×	1.32	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**
4×	5.28	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**
8×	21.12	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**	**[TBD]**

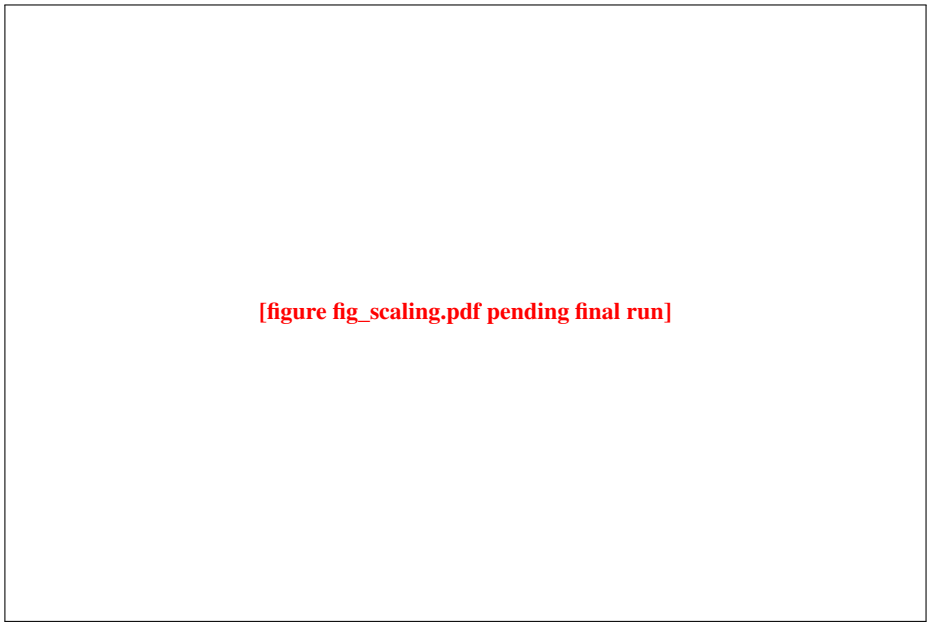


Fig. 4. Speedup over the serial build against image size for each thread count.

Code and data availability

The C source, the Kaggle driver, all CSV logs and the analysis scripts are in the project repository. DRIVE and STARE are available from their original providers [18,8].

Use of AI tools

Code, experiment scripts and the manuscript draft were produced with the assistance of an AI coding assistant (Claude, Anthropic) under the authors' direction. All numbers were measured by the authors' runs, and the authors reviewed and take responsibility for the content.

References

1. Argüello, F., Vilariño, D.L., Heras, D.B., Nieto, A.: GPU-based segmentation of retinal blood vessels. Journal of Real-Time Image Processing **14**(4), 773–782 (2018). https://doi.org/10.1007/s11554-014-0469-z
2. Ben Sayadia, S., Elloumi, Y., Akil, M., Bedoui, M.H.: Efficient graphical-processor-unit parallelization algorithm for computing Eigen values. Journal of Electronic Imaging **29**(6), 063008 (2020). https://doi.org/10.1117/1.JEI.29.6.063008
3. Chen, C., Chuah, J.H., Ali, R., Wang, Y.: Retinal vessel segmentation using deep learning: A review. IEEE Access **9**, 111985–112004 (2021). https://doi.org/10.1109/ACCESS.2021.3102176
4. Frangi, A.F., Niessen, W.J., Vincken, K.L., Viergever, M.A.: Multiscale vessel enhancement filtering. In: Medical Image Computing and Computer-Assisted Intervention (MICCAI'98). Lecture Notes in Computer Science, vol. 1496, pp. 130–137. Springer, Berlin, Germany (1998)
5. Fraz, M.M., Remagnino, P., Hoppe, A., Uyyanonvara, B., Rudnicka, A.R., Owen, C.G., Barman, S.A.: Blood vessel segmentation methodologies in retinal images – a survey. Computer Methods and Programs in Biomedicine **108**(1), 407–433 (2012). https://doi.org/10.1016/j.cmpb.2012.03.009
6. Galdran, A., Anjos, A., Dolz, J., Chakor, H., Lombaert, H., Ben Ayed, I.: State-of-the-art retinal vessel segmentation with minimalistic models. Scientific Reports **12**, 6174 (2022). https://doi.org/10.1038/s41598-022-09675-y
7. Gojić, G., Kundačina, O., Mišković, D., Dragan, D.: Overview of deep learning methods for retinal vessel segmentation. arXiv:2306.06116 (2023)
8. Hoover, A., Kouznetsova, V., Goldbaum, M.: Locating blood vessels in retinal images by piecewise threshold probing of a matched filter response. IEEE Transactions on Medical Imaging **19**(3), 203–210 (2000)
9. International Diabetes Federation: IDF diabetes atlas, 11th edition. https://diabetesatlas.org (2025), india country profile: 89.8 million adults aged 20–79 with diabetes in 2024
10. Karp, A.H., Flatt, H.P.: Measuring parallel processor performance. Communications of the ACM **33**(5), 539–543 (1990)
11. Mahapatra, S., Agrawal, S., Mishro, P.K., Pachori, R.B.: A novel framework for retinal vessel segmentation using optimal improved Frangi filter and adaptive weighted spatial FCM. Computers in Biology and Medicine **147**, 105770 (2022). https://doi.org/10.1016/j.compbiomed.2022.105770
12. Marr, D.T., Binns, F., Hill, D.L., Hinton, G., Koufaty, D.A., Miller, J.A., Upton, M.: Hyper-threading technology architecture and microarchitecture. Intel Technology Journal **6**(1), 4–15 (2002)
13. Mehmood, M., Iqbal, S., Khan, T.M., Spence, I., Fahim, M.: LVS-Net: A lightweight vessels segmentation network for retinal image analysis. arXiv:2412.05968 (2024)
14. Mehmood, M., Spence, I., Fahim, M.: LFA-Net: A lightweight network with LiteFusion attention for retinal vessel segmentation. arXiv:2509.21738 (2025)
15. Nahar, S., Wang, Z., Arkfeld, E., McMahon, G., Hakozaki, H., Schöneberg, J.: GPU-accelerated MitoGraph for high-throughput three-dimensional mitochondrial morphology analysis. BMC Bioinformatics **27**, 206 (2026). https://doi.org/10.1186/s12859-026-06441-z
16. Sobhan, S., Jalil, F.: LightVesselNet: An ultra-lightweight sub-100K parameter network for retinal blood vessel segmentation. arXiv:2606.05354 (2026)

17. Song, Y., Li, C., Xiao, S., Xiao, H., Guo, B.: Unsharp masking image enhancement the parallel algorithm based on cross-platform. Scientific Reports **12**, 20175 (2022). https://doi.org/10.1038/s41598-022-21745-9
18. Staal, J., Abràmoff, M.D., Niemeijer, M., Viergever, M.A., van Ginneken, B.: Ridge-based vessel segmentation in color images of the retina. IEEE Transactions on Medical Imaging **23**(4), 501–509 (2004)
19. Sule, O.O.: A survey of deep learning for retinal blood vessel segmentation methods: Taxonomy, trends, challenges and future directions. IEEE Access **10**, 38202–38236 (2022). https://doi.org/10.1109/ACCESS.2022.3163247
20. van der Walt, S., Schönberger, J.L., Nunez-Iglesias, J., Boulogne, F., Warner, J.D., Yager, N., Gouillart, E., Yu, T., the scikit-image contributors: scikit-image: Image processing in Python. PeerJ **2**, e453 (2014). https://doi.org/10.7717/peerj.453
21. Williams, S., Waterman, A., Patterson, D.: Roofline: An insightful visual performance model for multicore architectures. Communications of the ACM **52**(4), 65–76 (2009). https://doi.org/10.1145/1498765.1498785
22. Zuiderveld, K.: Contrast limited adaptive histogram equalization. In: Heckbert, P.S. (ed.) Graphics Gems IV, pp. 474–485. Academic Press, San Diego, CA, USA (1994)